

Intro to File IO



Learning Goals

- Read text from a file.
- Save changes to a file.

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
- **Package**: a collection of modules that can be accessed as a group using the package name.
- **import** : the Python keyword used to access data from other packages and modules inside of the current module.
- **PyPI**: the **Python Package Index**. A repository of Python packages that can be downloaded and made available to your application.
- **pip** : the command line tool used to download packages from PyPI. **pip** is installed on your computer automatically when you download Python.
- **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
- **Pipenv**: a virtual environment tool that uses **pip** to manage the modules, packages, and scripts that you intend to use in your application.
- **Mode** represents the file's permissions.

Introduction

When we work with computers, we need to do much more than execute code and browse the internet. If we want to save our work- or access data saved by other programs and computers- we need to use **files**.

A **file** is simply a container for storing information. This can be as plaintext in `.txt` files, or in special formatted files with `.docx` or `.pdf` extensions. There are also certain extensions- like `.py` - which give the computer and its users some hints about how to get the most out of the text contained inside, but which really label files that only store plaintext. For instance, you can open a `.py` file in an IDE like VSCode or execute it using the Python interpreter, but you can also open it in a simple text editor.

We've all worked with files many times before, but did you know that Python can open, write to, save, and close files as well? We call this process **file I/O** (input/output), and it's actually quite simple.

Opening a file

Before working with any file, we need to open it to access its contents. You can accomplish this in Python with the `open()` function:

```
text_file = open('file_directory/file_name.txt')
```

The only required argument for the open function is the path to the file in question. This can be an existing file or a path to a new one.

NOTE: Python can create files very easily, but it takes some extra work to create directories. It's usually best to make all of your directories before using the `open()` function.

Now we can use the `text_file` variable to do operations on the file.

What operations can do? It depends on the **mode** we have opened the file in. The default mode is **read**, but we can specify different modes using a second argument into the open function.

We can define the encoding of the file using the `encoding` attribute:

```
text_file = open('file_directory/file_name.txt', encoding='utf-8')
```

If you do not specify the encoding Python will use a default encoding. The default is platform dependent (whatever `locale.getpreferredencoding()` returns), though it is typically `utf-8`. It is considered best practice to always define the encoding.

Closing a file

A file should be closed as soon as we are done using it. An open file can block other programs from using the file depending on the mode we open it in. You can close the file by using the `close()` method:

```
text_file = open('file_name.txt', encoding='utf-8')
text_file.close()
```

Nobody is perfect. What if we forget to close a file? We can prevent problems like this by using a `with` statement:

```
with open('file_name.txt', encoding='utf-8') as text_file:
    text_file.read()
```

Inside the `with` code block we can use the variable `text_file`. When the block ends, Python will automatically call `text_file.close()`. The `with` statement guarantees the file will be closed even in the case of an exception.

Modes

The `mode` attribute of a file can tell you which mode the file had been opened in.

```
text_file = open('file_directory/file_name.txt')

print(text_file.mode)
# => r
```

The default mode is "read", but we can pass a second argument which allows us to open the file in many different modes that serve different use cases. These modes will allow us to do things like write to a file, open a file in binary, and append to a file.

Here are some example of how we can define a mode.

```
#append mode
text_file = open('file_directory/file_name.txt', mode='a', encoding='utf-8')

#write mode
text_file = open('file_directory/file_name.txt', mode='w' encoding='utf-8')
```

Reading from a file

When we want to read from a file we have to open it first. We can read from a file using the `read()` method:

```
# Lets say the text_file.txt contains the following sentence:
# The door swung open to reveal pink giraffes and red elephants.

text_file = open('text_file.txt', encoding='utf-8')
print(text_file.read())
# => The door swung open to reveal pink giraffes and red elephants.
```

What if we want to read from a super long file?

It is inefficient to hold the entire file in memory.

We can process one line at a time instead of using the `read()` method which gives us the entire file content.

```
with open('big_file.txt', encoding='utf-8') as text_file:
    for line in text_file:
        # Process the individual line
        print(line)
```

Writing to a File

Writing to a file is similar to reading from a file. The only difference is that we have to open the file in either the "write" or "append" mode.

- The write mode can be defined using the argument `mode='w'`
- The append mode can be defined using `mode='a'`

The write and append modes will create a new file if it does not already exist. We can use the `.write()` method to write and append to the file.

```
#Writing to a file
with open('log_file.txt', mode='w', encoding='utf-8') as log_file:
    log_file.write('Log 1')

with open('log_file.txt', mode='a', encoding='utf-8') as log_file:
    log_file.write('Log 2')
```

If a file already exists and we use the `mode='w'` the file will be overwritten.

If we use `mode='a'` we will append what we write to the existing file.

Conclusion

Now that we have a better understanding of how to interact with files in Python. We can apply this knowledge to many use cases like storing data, writing logs and interacting with other programs.

In the next lab, you'll get the chance to use the `open()` function to automate file I/O.

Resources

- [Python 3 Documentation](https://docs.python.org/3/)  [\(https://docs.python.org/3/\)](https://docs.python.org/3/)
- [Python File IO](https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files)  [\(https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files\)](https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files)